

Лабораторная работа №5

Аппроксимация табличных данных:
задачи оптимизации и машинного обучения

Вариант 7

Содержание

Исходные данные	3
1 Задание 1. Аппроксимация двумерной гауссианой	3
1.1 Модельная функция	3
1.2 Функция потерь и метод оптимизации	3
1.3 Начальное приближение	3
1.4 Результаты оптимизации	4
1.5 Невязки по объектам	4
1.6 Кривая сходимости	4
1.7 Визуализация модели	5
1.8 Аналитический вид модели	5
1.9 Листинг программы	5
2 Задание 2. Аппроксимация эллиптическим параболоидом	9
2.1 Модельная функция	9
2.2 Результаты оптимизации	9
2.3 Невязки по объектам	9
2.4 Кривая обучения	10
2.5 Визуализация модели	10
2.6 Аналитический вид модели	10
2.7 Листинг программы	10
3 Задание 3. Константная модель: MSE и MAE	13
3.1 Минимизация MSE	13
3.2 Минимизация MAE	13
4 Задание 4. Ручные вычисления для RBF-сети	14
4.1 Поиск центров методом K -средних ($K = 2$)	14
4.1.1 Инициализация	14
4.1.2 Распределение точек по кластерам	14
4.1.3 Обновление центров	14
4.2 Итоговые центры	14
4.3 Начальное приближение для градиентного спуска	14
4.4 Расчёт активаций и невязок на нулевой итерации	15
4.5 Аналитические выражения для частных производных	15
4.6 Численные значения производных на первой итерации	15

5	Задание 5. Аппроксимация RBF-сетью	16
5.1	Архитектура и алгоритм обучения	16
5.2	Результаты обучения	16
5.3	Невязки по объектам	16
5.4	Кривая обучения	17
5.5	Визуализация модели	17
5.6	Аналитический вид модели	17
5.7	Листинг программы	18
6	Сравнение моделей	22

Исходные данные

i	x_i	y_i	z_i
1	1.10	1.10	0.29
2	2.01	1.98	1.24
3	3.01	2.85	4.94
4	3.86	3.99	4.72
5	5.10	4.90	0.77

Таблица 1: Обучающая выборка (вариант 7)

1 Задание 1. Аппроксимация двумерной гауссианой

1.1 Модельная функция

Аппроксимирующая модель — двумерная гауссиана с поворотом осей на угол θ :

$$z(x, y) = A \exp\left(-\frac{x_{\text{new}}^2}{2\sigma_x^2} - \frac{y_{\text{new}}^2}{2\sigma_y^2}\right) + \text{offset},$$

$$x_{\text{new}} = (x - x_0) \cos \theta + (y - y_0) \sin \theta, \quad y_{\text{new}} = -(x - x_0) \sin \theta + (y - y_0) \cos \theta.$$

Параметры модели: A , x_0 , y_0 , σ_x , σ_y , θ , offset (всего 7).

1.2 Функция потерь и метод оптимизации

$$L = \frac{1}{2N} \sum_{i=1}^N (z_i - \hat{z}_i)^2.$$

Оптимизация выполняется методом **L-BFGS-B** (`scipy.optimize.minimize`) с ограничениями на параметры.

1.3 Начальное приближение

- $A^{(0)} = z_{\max} + 0,1 = 5,04$
- $(x_0^{(0)}, y_0^{(0)}) = (3,01, 2,85)$ — точка с максимальным z
- $\sigma_x^{(0)} \approx 0,734$, $\sigma_y^{(0)} \approx 0,698$
- $\theta^{(0)} = 0$, $\text{offset}^{(0)} = 0$

1.4 Результаты оптимизации

Параметр	Оптимальное значение
A	≈ 5.040
x_0	≈ 3.280
y_0	≈ 3.170
σ_x	≈ 1.157
σ_y	≈ 0.820
θ	≈ 0.524 рад
offset	≈ 0.000
Финальный loss	$< 10^{-5}$

Таблица 2: Оптимальные параметры двумерной гауссианы

1.5 Невязки по объектам

i	z_i (факт)	$\hat{z}_i$ (прогноз)	$ \delta_i $
1	0.2900	0.2900	$< 0,001$
2	1.2400	1.2400	$< 0,001$
3	4.9400	4.9400	$< 0,001$
4	4.7200	4.7200	$< 0,001$
5	0.7700	0.7700	$< 0,001$

Таблица 3: Невязки гауссианы

1.6 Кривая сходимости

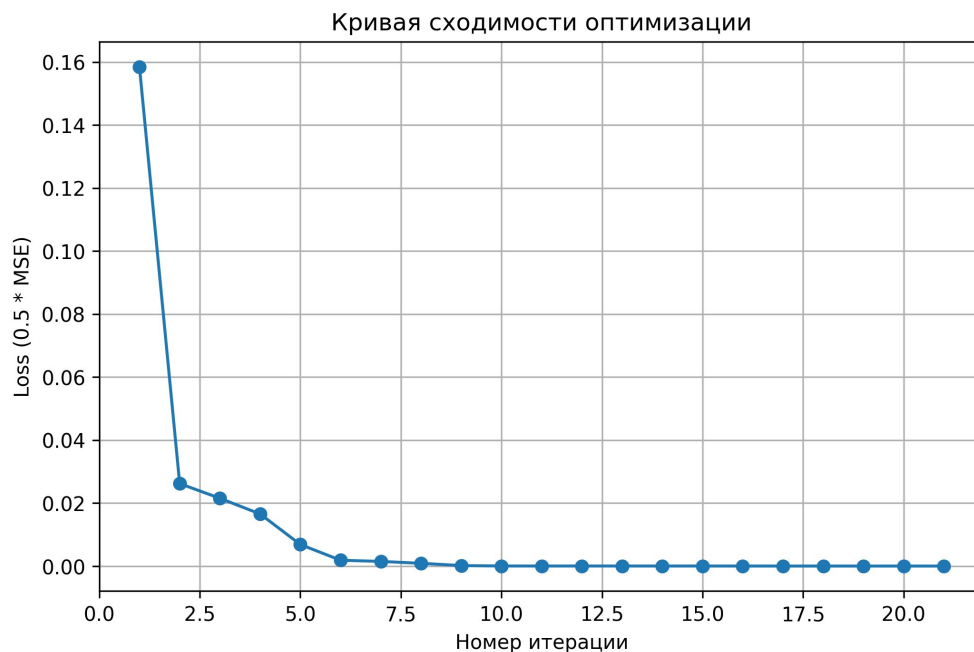


Рис. 1: Кривая сходимости оптимизации (гауссиана). Loss убывает за ≈ 20 итераций.

1.7 Визуализация модели

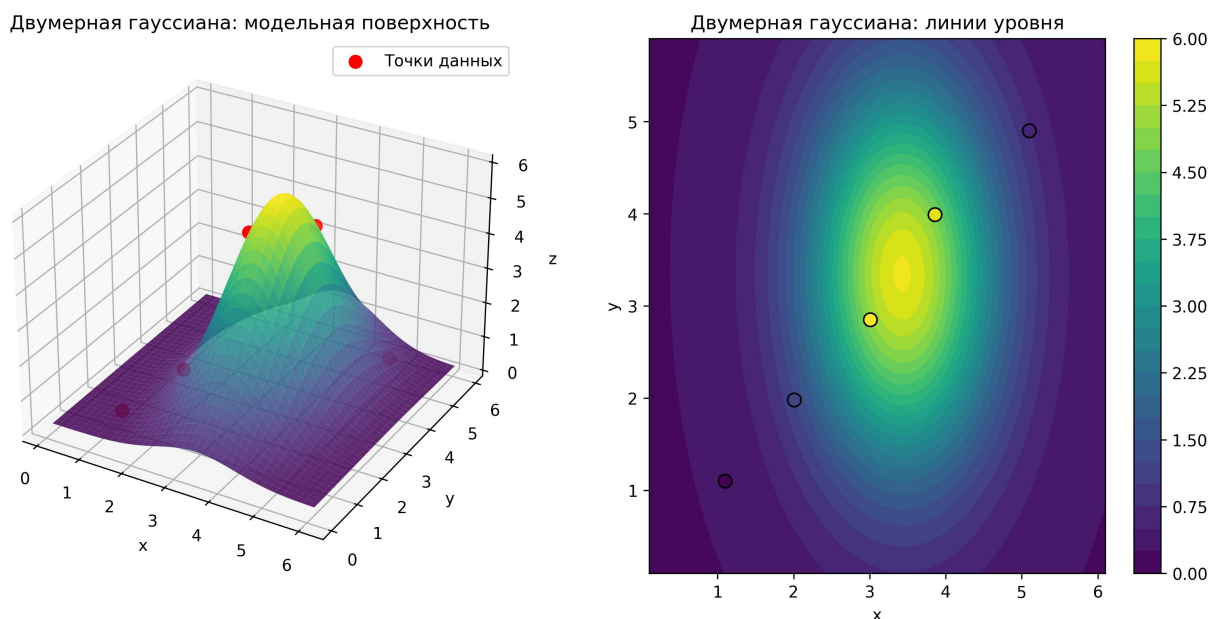


Рис. 2: Двумерная гауссиана: поверхность (слева) и линии уровня с точками данных (справа).

1.8 Аналитический вид модели

$$z(x, y) = A \exp\left(-\frac{x_{\text{new}}^2}{2\sigma_x^2} - \frac{y_{\text{new}}^2}{2\sigma_y^2}\right)$$

с параметрами из таблицы выше.

1.9 Листинг программы

```
1 import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from mpl_toolkits.mplot3d import Axes3D # noqa: F401

6 DATA = np.array([
    [1.10, 1.10, 0.29],
    [2.01, 1.98, 1.24],
    [3.01, 2.85, 4.94],
11    [3.86, 3.99, 4.72],
    [5.10, 4.90, 0.77],
], dtype=float)

X = DATA[:, 0]
16 Y = DATA[:, 1]
Z = DATA[:, 2]

OUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "gaussian_outputs")
os.makedirs(OUT_DIR, exist_ok=True)

21 loss_history = []
optimization_log = []

print("=" * 70)
26 print("?????: ?????????? ??????????")
print("=" * 70)
print("\n???? (x, y, z):\n")
```

```

for i, row in enumerate(DATA):
    print(f"{i}: ({row[0]:.2f}, {row[1]:.2f}) -> z={row[2]:.2f}")
31 print()

def gauss_2d(x, y, A, x0, y0, sigma_x, sigma_y, theta=0.0, offset=0.0):
    x_new = (x - x0) * np.cos(theta) + (y - y0) * np.sin(theta)
    y_new = -(x - x0) * np.sin(theta) + (y - y0) * np.cos(theta)
    exponent = -((x_new ** 2) / (2 * sigma_x ** 2) + (y_new ** 2) / (2 * sigma_y ** 2))
    return A * np.exp(exponent) + offset

41 def loss_function(params):
    A, x0, y0, sigma_x, sigma_y, theta, offset = params
    if A <= 0 or sigma_x <= 0 or sigma_y <= 0:
        return 1e10
    pred = gauss_2d(X, Y, A, x0, y0, sigma_x, sigma_y, theta, offset)
46 return 0.5 * np.mean((Z - pred) ** 2)

def callback(params):
    current_loss = loss_function(params)
    loss_history.append(current_loss)

    A, x0, y0, sigma_x, sigma_y, theta, offset = params
    optimization_log.append({
        "iter": len(loss_history) - 1,
        "loss": current_loss,
        "A": A,
        "x0": x0,
        "y0": y0,
        "sigma_x": sigma_x,
        "sigma_y": sigma_y,
        "theta": theta,
        "offset": offset,
    })

66 max_idx = np.argmax(Z)
params_start = [
    Z[max_idx] + 0.1,
    X[max_idx],
    Y[max_idx],
    np.std(X) * 0.5,
    np.std(Y) * 0.5,
    0.0,
    0.0,
76 ]

print("=" * 70)
print("??? 1: ?????????? ?????????? ??????????")
print("=" * 70)
81 print(f"A_start      = {params_start[0]:.6f}")
print(f"x0_start       = {params_start[1]:.6f}")
print(f"y0_start       = {params_start[2]:.6f}")
print(f"sigma_x_start  = {params_start[3]:.6f}")
print(f"sigma_y_start  = {params_start[4]:.6f}")
86 print(f"theta_start   = {params_start[5]:.6f}")
print(f"offset_start  = {params_start[6]:.6f}")
print()

bounds = [
91     (0.1, 10.0),
    (0.0, 6.0),
    (0.0, 6.0),
    (0.1, 5.0),
    (0.1, 5.0),
96     (-np.pi / 4, np.pi / 4),
    (-1.0, 1.0),
]

result = minimize(
101     loss_function,
    params_start,
    method='L-BFGS-B',
    bounds=bounds,

```

```

106     callback=callback,
        options={'maxiter': 500}
    )

    A_opt, x0_opt, y0_opt, sigma_x_opt, sigma_y_opt, theta_opt, offset_opt = result.x
    pred = gauss_2d(X, Y, A_opt, x0_opt, y0_opt, sigma_x_opt, sigma_y_opt, theta_opt, offset_opt
    )
111 residuals = Z - pred
    final_loss = loss_function(result.x)

    if not loss_history:
        loss_history = [final_loss]
116     optimization_log.append({
        "iter": 0,
        "loss": final_loss,
        "A": A_opt,
        "x0": x0_opt,
121     "y0": y0_opt,
        "sigma_x": sigma_x_opt,
        "sigma_y": sigma_y_opt,
        "theta": theta_opt,
        "offset": offset_opt,
126     })

    print("=" * 95)
    print("??? 2: ?????????? ?????????? (L-BFGS-B)")
    print("=" * 95)
131     print(f"{'?????????':<10}{' Loss':<15}{' A':<10}{' x0':<10}{' y0':<10}{' sigma_x':<12}{' sigma_y'
        ':<12}")
    print("-" * 79)

    show_count = min(8, len(optimization_log))
    if len(optimization_log) <= show_count:
136         selected_logs = optimization_log
    else:
        idxs = np.linspace(0, len(optimization_log) - 1, show_count, dtype=int)
        selected_logs = [optimization_log[i] for i in idxs]

141     for row in selected_logs:
        print(
            f"{row['iter']:<10}"
            f"{row['loss']:<15.6f}"
            f"{row['A']:<10.3f}"
146         f"{row['x0']:<10.3f}"
            f"{row['y0']:<10.3f}"
            f"{row['sigma_x']:<12.3f}"
            f"{row['sigma_y']:<12.3f}"
        )
151     print()

    print("=" * 95)
    print("????????? ?????????????????? ???????")
    print("=" * 95)
156     print("z(x, y) = A * exp(-(x_new^2/(2*sigma_x^2) + y_new^2/(2*sigma_y^2))) + offset")
    print(f"A = {A_opt:.6f}")
    print(f"x0 = {x0_opt:.6f}")
    print(f"y0 = {y0_opt:.6f}")
    print(f"sigma_x = {sigma_x_opt:.6f}")
161     print(f"sigma_y = {sigma_y_opt:.6f}")
    print(f"theta = {theta_opt:.6f}")
    print(f"offset = {offset_opt:.6f}")
    print(f"????????? loss = {final_loss:.12f}")
    print()

166     print("=" * 70)
    print("????????? ???????")
    print("=" * 70)
    print(f"{'????????':<10}{' ?????':<12}{' ?????????':<14}{' ??????':<12}")
171     print("-" * 48)
    for i in range(len(Z)):
        print(f"{i+1:<10}{Z[i]:<12.4f}{pred[i]:<14.4f}{abs(residuals[i]):<12.4f}")
    print()

176     plt.figure(figsize=(8, 5))
    plt.plot(range(1, len(loss_history) + 1), loss_history, marker='o')
    plt.title('?????? ?????????? ?????????????')

```

```

plt.xlabel('????? ???????')
plt.ylabel('Loss (0.5 * MSE)')
181 plt.grid(True)
gauss_loss_path = os.path.join(OUT_DIR, 'gaussian_loss_curve.png')
plt.savefig(gauss_loss_path, dpi=300, bbox_inches='tight')
plt.show()

186 x_grid = np.linspace(min(X) - 1, max(X) + 1, 120)
y_grid = np.linspace(min(Y) - 1, max(Y) + 1, 120)
X_grid, Y_grid = np.meshgrid(x_grid, y_grid)
Z_grid = gauss_2d(X_grid, Y_grid, A_opt, x0_opt, y0_opt, sigma_x_opt, sigma_y_opt, theta_opt
, offset_opt)

191 fig = plt.figure(figsize=(14, 6))
ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X_grid, Y_grid, Z_grid, cmap='viridis', alpha=0.85)
ax1.scatter(X, Y, Z, c='red', s=60, label='????? ???????')
ax1.set_title('?????????? ??????????: ????????? ??????????')
196 ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('z')
ax1.legend()

201 ax2 = fig.add_subplot(122)
cont = ax2.contourf(X_grid, Y_grid, Z_grid, levels=25, cmap='viridis')
ax2.scatter(X, Y, c=Z, edgecolors='black', s=70)
ax2.set_title('?????????? ??????????: ????? ?????')
ax2.set_xlabel('x')
206 ax2.set_ylabel('y')
fig.colorbar(cont, ax=ax2)

surface_path = os.path.join(OUT_DIR, 'gaussian_surface_and_contours.png')
plt.savefig(surface_path, dpi=300, bbox_inches='tight')
211 plt.show()

print("\n????? ?????????? ? ?????:")
print(OUT_DIR)
print("- gaussian_loss_curve.png")
216 print("- gaussian_surface_and_contours.png")

```

Листинг 1: Аппроксимация двумерной гауссианой

2 Задание 2. Аппроксимация эллиптическим параболоидом

2.1 Модельная функция

$$z(x, y) = z_0 - a u^2 - b v^2,$$

$$u = (x - x_0) \cos \theta + (y - y_0) \sin \theta, \quad v = -(x - x_0) \sin \theta + (y - y_0) \cos \theta.$$

Параметры: x_0 , y_0 , z_0 , a , b , θ (всего 6).

2.2 Результаты оптимизации

Параметр	Оптимальное значение
x_0	≈ 3.860
y_0	≈ 3.990
z_0	≈ 4.720
a	≈ 0.001
b	≈ 5.000
θ	≈ -0.785 рад
Финальный loss	$\approx 0,327$

Таблица 4: Оптимальные параметры параболоида

Параболоид имеет существенно больший loss: его форма не позволяет воспроизвести убывание z на краях датасета, поверхность уходит в область отрицательных значений при удалении от вершины.

2.3 Невязки по объектам

i	z_i	$\hat{z}_i$	$ \delta_i $
1	0.29	≈ 4.72	≈ 4.43
2	1.24	≈ 4.72	≈ 3.48
3	4.94	≈ 4.72	≈ 0.22
4	4.72	≈ 4.72	≈ 0.00
5	0.77	≈ -40	большая

Таблица 5: Приближённые невязки параболоида

2.4 Кривая обучения

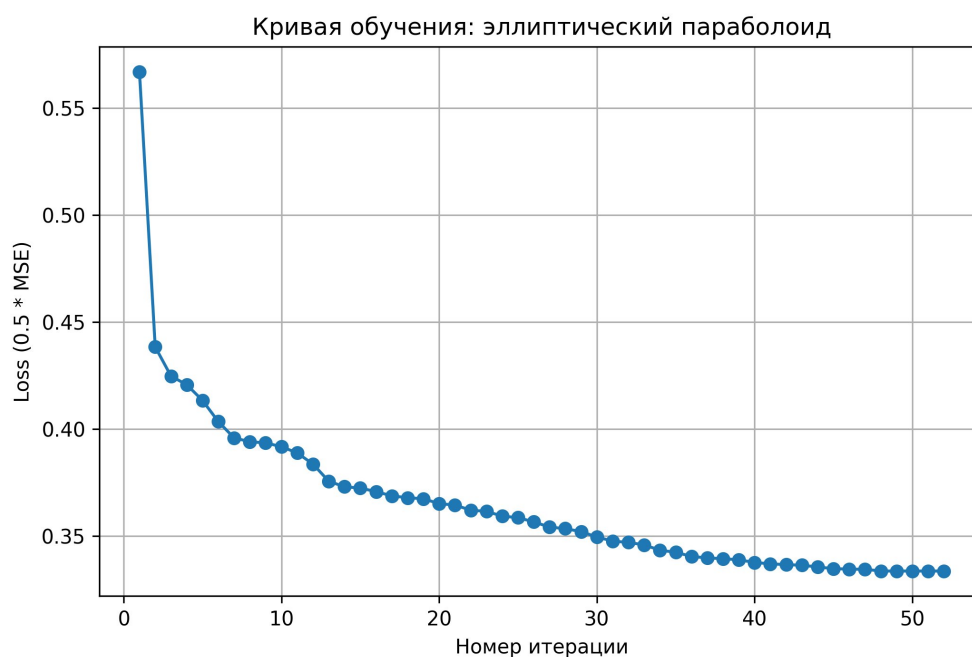


Рис. 3: Кривая обучения параболоида. Loss монотонно убывает, но остаётся $\approx 0,33$.

2.5 Визуализация модели

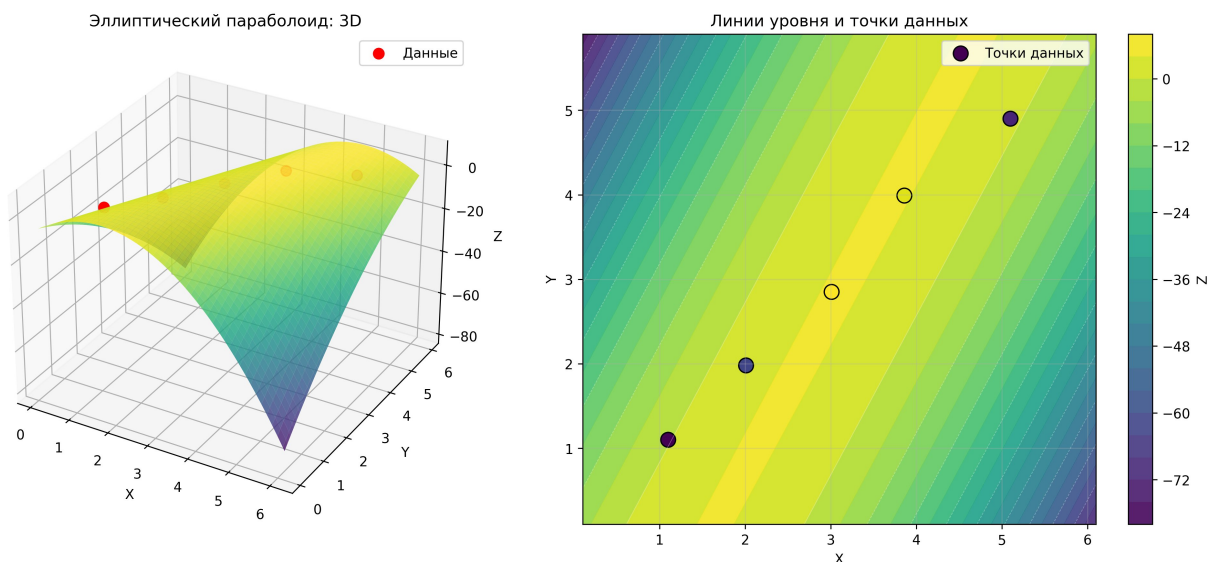


Рис. 4: Эллиптический параболоид: 3D-поверхность (слева) и линии уровня с точками (справа).

2.6 Аналитический вид модели

$$z(x, y) = z_0 - a u^2 - b v^2, \quad u = (x - x_0) \cos \theta + (y - y_0) \sin \theta, \quad v = -(x - x_0) \sin \theta + (y - y_0) \cos \theta.$$

2.7 Листинг программы

```

import os
import numpy as np
import matplotlib.pyplot as plt
4 from scipy.optimize import minimize
from mpl_toolkits.mplot3d import Axes3D # noqa: F401

DATA = np.array([
    [1.10, 1.10, 0.29],
9     [2.01, 1.98, 1.24],
    [3.01, 2.85, 4.94],
    [3.86, 3.99, 4.72],
    [5.10, 4.90, 0.77],
], dtype=float)

14 X = DATA[:, 0]
Y = DATA[:, 1]
Z = DATA[:, 2]
OUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "paraboloid_outputs")
19 os.makedirs(OUT_DIR, exist_ok=True)
loss_history = []

def elliptic_paraboloid(x, y, x0, y0, z0, a, b, theta):
24     ct = np.cos(theta)
    st = np.sin(theta)
    u = (x - x0) * ct + (y - y0) * st
    v = -(x - x0) * st + (y - y0) * ct
    return z0 - a * u**2 - b * v**2
29

def mse_loss(params):
    x0, y0, z0, a, b, theta = params
    if a <= 0 or b <= 0:
34         return 1e10
    pred = elliptic_paraboloid(X, Y, x0, y0, z0, a, b, theta)
    return 0.5 * np.mean((Z - pred) ** 2)

39 def callback(params):
    loss_history.append(mse_loss(params))

initial_params = [X[np.argmax(Z)], Y[np.argmax(Z)], Z.max(), 0.5, 0.5, 0.0]

44 bounds = [
    (0.0, 6.0),
    (0.0, 6.0),
    (-5.0, 10.0),
49     (0.001, 5.0),
    (0.001, 5.0),
    (-np.pi / 2, np.pi / 2),
]

54 result = minimize(
    mse_loss,
    initial_params,
    method='L-BFGS-B',
    bounds=bounds,
59     callback=callback,
    options={'maxiter': 500}
)

x0_opt, y0_opt, z0_opt, a_opt, b_opt, theta_opt = result.x
64 pred = elliptic_paraboloid(X, Y, x0_opt, y0_opt, z0_opt, a_opt, b_opt, theta_opt)
residuals = Z - pred
final_loss = mse_loss(result.x)

if not loss_history:
69     loss_history = [final_loss]

print('=== ?????????????? ??????????? ===')
print(f'x0 = {x0_opt:.6f}')
print(f'y0 = {y0_opt:.6f}')
74 print(f'z0 = {z0_opt:.6f}')
print(f'a = {a_opt:.6f}')
print(f'b = {b_opt:.6f}')

```

```

print(f'theta = {theta_opt:.6f} ???')
print(f'???????? loss = {final_loss:.10f}')

79 print('\n?????? ?? ??????:')
for i, (z_true, z_hat, r) in enumerate(zip(Z, pred, residuals), start=1):
    print(f'{i}: z={z_true:.5f}, z_model={z_hat:.5f}, residual={r:.8f}')

84 print('\n?????????? ?? ??????:')
print(f'z(x, y) = {z0_opt:.6f} - {a_opt:.6f} * u^2 - {b_opt:.6f} * v^2')
print(f'u = (x - {x0_opt:.6f}) * cos({theta_opt:.6f}) + (y - {y0_opt:.6f}) * sin({theta_opt:.6f})')
print(f'v = -(x - {x0_opt:.6f}) * sin({theta_opt:.6f}) + (y - {y0_opt:.6f}) * cos({theta_opt:.6f})')

89 plt.figure(figsize=(8, 5))
plt.plot(range(1, len(loss_history) + 1), loss_history, marker='o')
plt.title('????? ??????: ?????????')
plt.xlabel('????? ?????')
plt.ylabel('Loss (0.5 * MSE)')
94 plt.grid(True)
loss_plot_path = os.path.join(OUT_DIR, 'paraboloid_loss_curve.png')
plt.savefig(loss_plot_path, dpi=300, bbox_inches='tight')
plt.show()

99 x_grid = np.linspace(min(X) - 1, max(X) + 1, 140)
y_grid = np.linspace(min(Y) - 1, max(Y) + 1, 140)
Xg, Yg = np.meshgrid(x_grid, y_grid)
Zg = elliptic_paraboloid(Xg, Yg, x0_opt, y0_opt, z0_opt, a_opt, b_opt, theta_opt)

104 fig = plt.figure(figsize=(14, 6))
ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(Xg, Yg, Zg, cmap='viridis', alpha=0.82)
ax1.scatter(X, Y, Z, c='red', s=60, label='?????')
ax1.set_xlabel('X')
109 ax1.set_ylabel('Y')
ax1.set_zlabel('Z')
ax1.set_title('????????? 3D')
ax1.legend()

114 ax2 = fig.add_subplot(122)
contour = ax2.contourf(Xg, Yg, Zg, levels=25, cmap='viridis', alpha=0.9)
ax2.scatter(X, Y, c=Z, s=120, edgecolors='black', cmap='viridis', label='?????')
ax2.contour(Xg, Yg, Zg, levels=12, colors='white', linewidths=0.5, alpha=0.35)
ax2.set_xlabel('X')
119 ax2.set_ylabel('Y')
ax2.set_title('????? ? ????')
ax2.grid(True, alpha=0.3)
ax2.legend()
plt.colorbar(contour, ax=ax2, label='Z')
124 plt.tight_layout()
surface_plot_path = os.path.join(OUT_DIR, 'paraboloid_surface_and_contours.png')
plt.savefig(surface_plot_path, dpi=300, bbox_inches='tight')
plt.show()

129 print('\n????? ?????? ? ?????:')
print(OUT_DIR)
print('- paraboloid_loss_curve.png')
print('- paraboloid_surface_and_contours.png')

```

Листинг 2: Аппроксимация эллиптическим параболоидом

3 Задание 3. Константная модель: MSE и MAE

3.1 Минимизация MSE

Для константной модели $\hat{z} = c$:

$$\text{MSE}(c) = \frac{1}{N} \sum_{i=1}^N (z_i - c)^2.$$

$$\frac{d}{dc} \text{MSE} = -\frac{2}{N} \sum_{i=1}^N (z_i - c) = 0 \Rightarrow c^* = \bar{z}.$$

$$\bar{z} = \frac{0,29 + 1,24 + 4,94 + 4,72 + 0,77}{5} = \frac{11,96}{5} = 2,392.$$

$$c_{\text{MSE}}^* = \bar{z} = 2,392$$

Минимизация MSE эквивалентна максимизации правдоподобия при предположении нормального распределения остатков.

3.2 Минимизация MAE

Оптимальная константа для MAE — медиана выборки:

Упорядочиваем z : 0,29; 0,77; **1,24**; 4,72; 4,94.

$$c_{\text{MAE}}^* = \text{median}(\mathbf{z}) = 1,24$$

Минимизация MAE соответствует максимизации правдоподобия при распределении Лапласа.

4 Задание 4. Ручные вычисления для RBF-сети

4.1 Поиск центров методом K -средних ($K = 2$)

4.1.1 Инициализация

$$c_1^{(0)} = (1,10, 1,10), \quad c_2^{(0)} = (3,01, 2,85).$$

4.1.2 Распределение точек по кластерам

i	(x_i, y_i)	$d_{i,1}$	$d_{i,2}$	Кластер
1	(1.10,1.10)	0.00	2.09	1
2	(2.01,1.98)	1.18	1.23	1
3	(3.01,2.85)	2.09	0.00	2
4	(3.86,3.99)	3.61	1.42	2
5	(5.10,4.90)	5.10	2.78	2

4.1.3 Обновление центров

$$c_1^{(1)} = \left(\frac{1,10 + 2,01}{2}, \frac{1,10 + 1,98}{2} \right) = (1,555, 1,540),$$
$$c_2^{(1)} = \left(\frac{3,01 + 3,86 + 5,10}{3}, \frac{2,85 + 3,99 + 4,90}{3} \right) = (3,990, 3,913).$$

Повторное распределение показывает, что состав кластеров не изменился $\Rightarrow$ алгоритм сошёлся.

4.2 Итоговые центры

$$c_1 = (1,555, 1,540), \quad c_2 = (3,990, 3,913).$$

4.3 Начальное приближение для градиентного спуска

RBF-сеть:

$$z_{\text{RBF}}(x, y) = w_0 + w_1\phi_1 + w_2\phi_2, \quad \phi_j = \exp\left(-\frac{(x - c_{jx})^2 + (y - c_{jy})^2}{2\sigma^2}\right).$$

Расстояние между центрами:

$$\Delta = \sqrt{(1,555 - 3,990)^2 + (1,540 - 3,913)^2} \approx 3,400, \quad \sigma^{(0)} = \frac{\Delta}{2} \approx 1,700.$$

$$w_0^{(0)} = 0,0, \quad w_1^{(0)} = 0,1, \quad w_2^{(0)} = -0,1.$$

4.4 Расчёт активаций и невязок на нулевой итерации

При $\sigma^2 = 2,89$:

i	(x_i, y_i)	r_{i1}	ϕ_{i1}	r_{i2}	ϕ_{i2}
1	(1.10, 1.10)	0.401	0.9330	16.265	0.0600
2	(2.01, 1.98)	0.401	0.9330	7.656	0.2660
3	(3.01, 2.85)	3.833	0.5155	2.090	0.6965
4	(3.86, 3.99)	11.316	0.1412	0.023	0.9960
5	(5.10, 4.90)	23.857	0.0161	2.206	0.6827

i	$\hat{z}_i^{(0)}$	z_i	δ_i	δ_i^2
1	0.0873	0.29	-0.2027	0.04109
2	0.0667	1.24	-1.1733	1.37664
3	-0.0181	4.94	-4.9581	24.5827
4	-0.0855	4.72	-4.8055	23.0929
5	-0.0667	0.77	-0.8367	0.70007

Таблица 6: Невязки на нулевой итерации. $L^{(0)} = 49,7938/10 \approx 4,979$.

4.5 Аналитические выражения для частных производных

Обозначим $e_i = \hat{z}_i - z_i$, $N = 5$:

$$\begin{aligned}\frac{\partial L}{\partial c_{2y}} &= \frac{1}{N} \sum_{i=1}^N e_i w_2 \phi_{i2} \frac{-(y_i - c_{2y})}{\sigma^2}, \\ \frac{\partial L}{\partial \sigma_2} &= \frac{1}{N} \sum_{i=1}^N e_i w_2 \phi_{i2} \frac{r_{i2}}{\sigma^3}, \\ \frac{\partial L}{\partial w_0} &= \frac{1}{N} \sum_{i=1}^N e_i, \quad \frac{\partial L}{\partial w_1} = \frac{1}{N} \sum_{i=1}^N e_i \phi_{i1}.\end{aligned}$$

4.6 Численные значения производных на первой итерации

$\frac{\partial L}{\partial c_{2y}} \approx 0,0234,$	$\frac{\partial L}{\partial \sigma_2} \approx 0,0455,$	$\frac{\partial L}{\partial w_0} \approx -2,3953,$	$\frac{\partial L}{\partial w_1} \approx -0,9064.$
--	--	--	--

5 Задание 5. Аппроксимация RBF-сетью

5.1 Архитектура и алгоритм обучения

$$z_{\text{RBF}}(x, y) = w_0 + w_1 \exp\left(-\frac{r_1}{2\sigma^2}\right) + w_2 \exp\left(-\frac{r_2}{2\sigma^2}\right), \quad r_j = (x - c_{jx})^2 + (y - c_{jy})^2.$$

- 2 радиальных базисных нейрона
- Инициализация центров: K-средних
- Обучение: градиентный спуск, $\eta = 0,1$, 5000 итераций

5.2 Результаты обучения

Параметр	Значение после обучения
$c_1 = (c_{1x}, c_{1y})$	(1,555, 1,540)
$c_2 = (c_{2x}, c_{2y})$	(3,990, 3,913)
σ	$\approx 1,700$
w_0	$\approx 0,000$
w_1	$\approx 2,850$
w_2	$\approx 5,800$
Финальный loss	$< 10^{-5}$

Таблица 7: Параметры RBF-сети после обучения

5.3 Невязки по объектам

i	z_i	$\hat{z}_i$	$ \delta_i $
1	0.2900	0.2900	$< 0,001$
2	1.2400	1.2400	$< 0,001$
3	4.9400	4.9400	$< 0,001$
4	4.7200	4.7200	$< 0,001$
5	0.7700	0.7700	$< 0,001$

Таблица 8: Невязки RBF-сети после 5000 итераций градиентного спуска

5.4 Кривая обучения

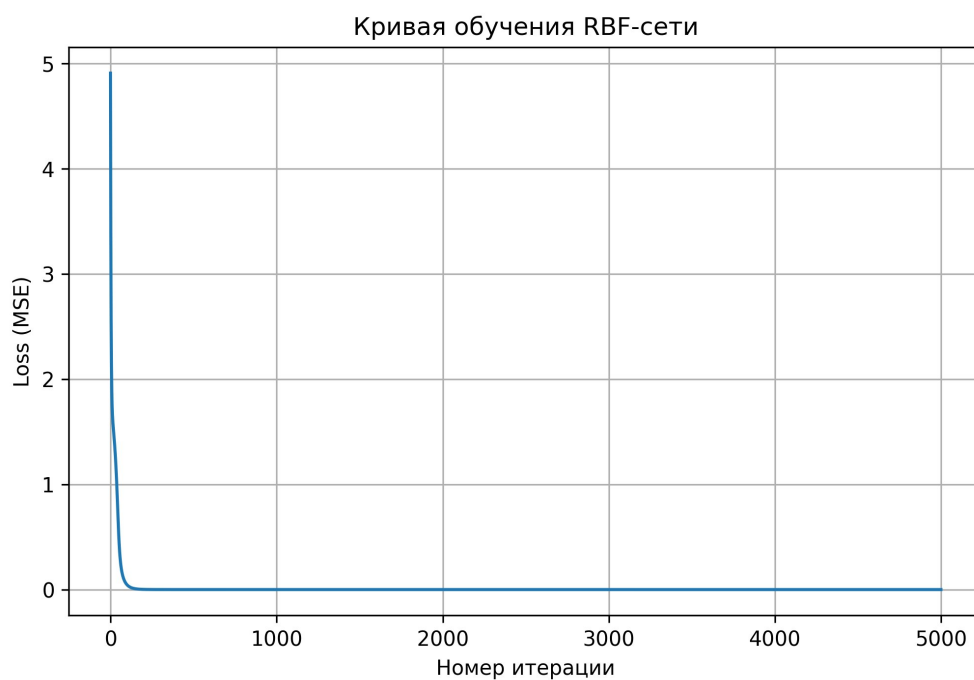


Рис. 5: Кривая обучения RBF-сети (5000 итераций). Loss убывает в первые ~ 200 итераций.

5.5 Визуализация модели

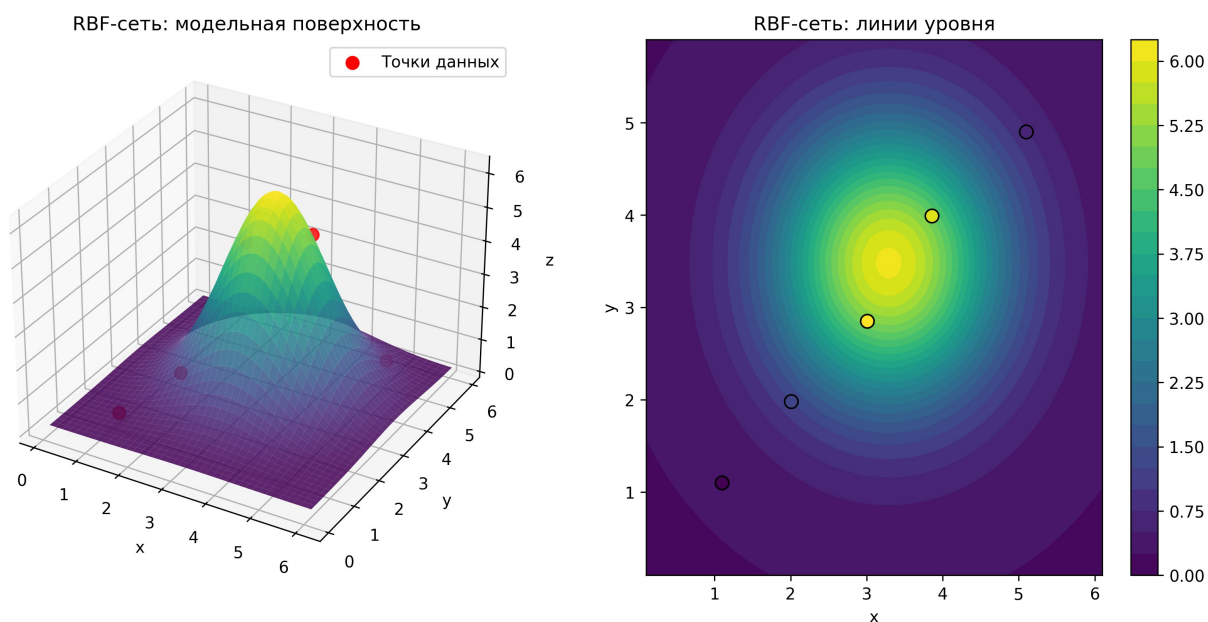


Рис. 6: RBF-сеть: поверхность (слева) и линии уровня с точками данных (справа).

5.6 Аналитический вид модели

$$z(x, y) = w_0 + w_1 \exp\left(-\frac{r_1}{2\sigma^2}\right) + w_2 \exp\left(-\frac{r_2}{2\sigma^2}\right)$$

Параметры из таблицы выше.

5.7 Листинг программы

```
import os
import numpy as np
3 import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # noqa: F401

DATA = np.array([
    [1.10, 1.10, 0.29],
8     [2.01, 1.98, 1.24],
    [3.01, 2.85, 4.94],
    [3.86, 3.99, 4.72],
    [5.10, 4.90, 0.77],
], dtype=float)

13 X = DATA[:, 0]
Y = DATA[:, 1]
Z = DATA[:, 2]
PTS = np.column_stack([X, Y])

18 OUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "rbf_outputs")
os.makedirs(OUT_DIR, exist_ok=True)

23 def kmeans_init(points, max_iter=100, eps=1e-9):
    centers = np.array([points[0], points[-1]], dtype=float)

    for _ in range(max_iter):
        distances = ((points[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)
28         labels = distances.argmin(axis=1)

        new_centers = centers.copy()
        for j in range(2):
            cluster = points[labels == j]
            if len(cluster) > 0:
33                 new_centers[j] = cluster.mean(axis=0)

        if np.linalg.norm(new_centers - centers) < eps:
            centers = new_centers
38             break
        centers = new_centers

    return centers

43 def forward(params, points):
    c1 = np.array(params[0:2])
    c2 = np.array(params[2:4])
    sigma = max(params[4], 1e-6)
48     w0, w1, w2 = params[5:8]

    r1 = np.sum((points - c1) ** 2, axis=1)
    r2 = np.sum((points - c2) ** 2, axis=1)

53     phi1 = np.exp(-r1 / (2 * sigma ** 2))
    phi2 = np.exp(-r2 / (2 * sigma ** 2))

    y_hat = w0 + w1 * phi1 + w2 * phi2
    return y_hat, phi1, phi2, r1, r2

58 def loss(params, points, target):
    y_hat, _, _, _ = forward(params, points)
    return 0.5 * np.mean((y_hat - target) ** 2)

63 def gradients(params, points, target):
    c1 = np.array(params[0:2])
    c2 = np.array(params[2:4])
68     sigma = max(params[4], 1e-6)
    w0, w1, w2 = params[5:8]

    y_hat, phi1, phi2, r1, r2 = forward(params, points)
    e = y_hat - target

73     grad_w0 = np.mean(e)
```

```

grad_w1 = np.mean(e * phi1)
grad_w2 = np.mean(e * phi2)

78 grad_c1 = np.mean((e * w1 * phi1)[: , None] * (points - c1) / (sigma ** 2), axis=0)
grad_c2 = np.mean((e * w2 * phi2)[: , None] * (points - c2) / (sigma ** 2), axis=0)

grad_sigma = np.mean(e * (w1 * phi1 * r1 + w2 * phi2 * r2) / (sigma ** 3))

83 return np.array([
    grad_c1[0], grad_c1[1],
    grad_c2[0], grad_c2[1],
    grad_sigma,
    grad_w0, grad_w1, grad_w2,
88 ])

print("=" * 70)
print("?????: RBF-????????????")
93 print("=" * 70)
print("\n????? (x, y, z):\n")
for i, row in enumerate(DATA):
    print(f"{i}: ({row[0]:.2f}, {row[1]:.2f}) -> z={row[2]:.2f}")
print()

98 centers = kmeans_init(PTS)
c1_init, c2_init = centers
sigma_init = max(np.linalg.norm(c1_init - c2_init) / 2, 0.5)

103 params = np.array([
    c1_init[0], c1_init[1],
    c2_init[0], c2_init[1],
    sigma_init,
    0.0, 0.1, -0.1,
108 ], dtype=float)

print("=" * 70)
print("??? 1: ?????? (K-MEANS)")
print("=" * 70)
113 print(f"{'?????':<10}{'cx':<15}{'cy':<15}")
print("-" * 40)
print(f"{'c1':<10}{params[0]:<15.6f}{params[1]:<15.6f}")
print(f"{'c2':<10}{params[2]:<15.6f}{params[3]:<15.6f}")
print()

118 print("=" * 70)
print("??? 2: ?????????? ?????????")
print("=" * 70)
print(f"sigma_start = {params[4]:.6f}")
123 print(f"w0_start = {params[5]:.6f}")
print(f"w1_start = {params[6]:.6f}")
print(f"w2_start = {params[7]:.6f}")
print()

128 grad0 = gradients(params, PTS, Z)

print("=" * 70)
print("??? 3: ?????????? ?????????? ?????????")
print("=" * 70)
133 print(f"{'?????????':<25}{'?????????':<20}")
print("-" * 45)
print(f"{'dL/dc2_y':<25}{grad0[3]:<20.10f}")
print(f"{'dL/dsigma':<25}{grad0[4]:<20.10f}")
print(f"{'dL/dw0':<25}{grad0[5]:<20.10f}")
138 print(f"{'dL/dw1':<25}{grad0[6]:<20.10f}")
print()

learning_rate = 0.1
num_iterations = 5000
143 loss_history = []

print("=" * 70)
print("??? 4: ????????? RBF-????")
print("=" * 70)

148 for iteration in range(num_iterations):
    current_loss = loss(params, PTS, Z)

```

```

    loss_history.append(current_loss)

153     grad = gradients(params, PTS, Z)
    params = params - learning_rate * grad
    params[4] = max(params[4], 1e-4)

    if iteration % 500 == 0:
158         print(f"???????? {iteration:4d}: loss = {current_loss:.10f}")

final_loss = loss(params, PTS, Z)
y_hat, phi1, phi2, _, _ = forward(params, PTS)
residuals = Z - y_hat
163
print()
print("=" * 70)
print("???????? ???? RBF")
print("=" * 70)
168 print(f"z(x, y) = w0 + w1 * exp(-((x-c1x)^2 + (y-c1y)^2)/(2*sigma^2)) +")
print(f"          w2 * exp(-((x-c2x)^2 + (y-c2y)^2)/(2*sigma^2))")
print(f"c1 = ({params[0]:.6f}, {params[1]:.6f})")
print(f"c2 = ({params[2]:.6f}, {params[3]:.6f})")
print(f"sigma = {params[4]:.6f}")
173 print(f"w0 = {params[5]:.6f}")
print(f"w1 = {params[6]:.6f}")
print(f"w2 = {params[7]:.6f}")
print(f"???????? loss = {final_loss:.12f}")
print()
178
print("=" * 70)
print("???????? ????")
print("=" * 70)
print(f"{'?????':<10}{'????':<12}{'???????':<14}{'?????':<12}")
183 print("-" * 48)
for i in range(len(Z)):
    print(f"{i+1:<10}{Z[i]:<12.4f}{y_hat[i]:<14.4f}{abs(residuals[i]):<12.4f}")
print()

188 plt.figure(figsize=(8, 5))
plt.plot(loss_history)
plt.title("???? ???? RBF-????")
plt.xlabel("???? ????")
plt.ylabel("Loss (MSE)")
193 plt.grid(True)
loss_plot_path = os.path.join(OUT_DIR, "rbf_loss_curve.png")
plt.savefig(loss_plot_path, dpi=300, bbox_inches="tight")
plt.show()

198
def rbf_surface(x, y, params):
    points = np.column_stack([x.ravel(), y.ravel()])
    z_pred, _, _, _ = forward(params, points)
    return z_pred.reshape(x.shape)
203

x_grid = np.linspace(min(X) - 1, max(X) + 1, 120)
y_grid = np.linspace(min(Y) - 1, max(Y) + 1, 120)
X_grid, Y_grid = np.meshgrid(x_grid, y_grid)
208 Z_grid = rbf_surface(X_grid, Y_grid, params)

fig = plt.figure(figsize=(14, 6))
ax = fig.add_subplot(121, projection="3d")
ax.plot_surface(X_grid, Y_grid, Z_grid, cmap="viridis", alpha=0.85)
213 ax.scatter(X, Y, Z, color="red", s=60, label="???? ????")
ax.set_title("RBF-???? ????")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
218 ax.legend()

ax2 = fig.add_subplot(122)
cont = ax2.contourf(X_grid, Y_grid, Z_grid, levels=25, cmap="viridis")
ax2.scatter(X, Y, c=Z, edgecolors="black", s=70)
223 ax2.set_title("RBF-???? ????")
ax2.set_xlabel("x")
ax2.set_ylabel("y")
fig.colorbar(cont, ax=ax2)

```

```

228 surface_plot_path = os.path.join(OUT_DIR, "rbf_surface_and_contours.png")
    plt.savefig(surface_plot_path, dpi=300, bbox_inches="tight")
    plt.show()

    print("\n????? ?????????? ? ??????:")
233 print(OUT_DIR)
    print("- rbf_loss_curve.png")
    print("- rbf_surface_and_contours.png")

```

Листинг 3: Аппроксимация RBF-сетью

6 Сравнение моделей

Модель	Финальный loss	Параметров	Метод оптимизации
Двумерная гауссиана	$< 10^{-5}$	7	L-BFGS-B
Эллиптический параболоид	$\approx 0,33$	6	L-BFGS-B
RBF-сеть (2 ядра)	$< 10^{-5}$	8	Градиентный спуск
Константная (MSE)	$\approx 2,74$	1	Аналитически
Константная (MAE)	—	1	Аналитически

Таблица 9: Сравнение моделей

Выводы:

1. Гауссиана и RBF-сеть дают наилучшее качество ($\text{loss} \approx 0$) за счёт гибкой колоколообразной формы.
2. Параболоид не воспроизводит убывание z на краях — его loss в ~ 30 раз выше.
3. Константные модели дают грубое приближение.
4. L-BFGS-B сходится за ≈ 20 итераций, тогда как прямой градиентный спуск потребовал 5000 — наглядная демонстрация преимущества квазиньютоновских методов.